

MovieTheater.API

A comprehensive movie theater management system built with .NET 8, featuring booking management, payment processing, real-time seat selection, and administrative tools.

Architecture

This project follows Clean Architecture principles with the following layers:

- **MovieTheater.API** - Web API layer with controllers and configuration
- **MovieTheater.Application** - Business logic, services, and SignalR hubs
- **MovieTheater.Domain** - Entities, DTOs, and domain models
- **MovieTheater.Infrastructure** - Data access, repositories, and external services
- **MovieTheater.UnitTest** - Unit test suite

Features

- **User Management** - Registration, authentication, JWT tokens, role-based access
- **Movie Management** - CRUD operations, scheduling, metadata management
- **Booking System** - Real-time seat selection, booking management, expiry handling
- **Payment Integration** - Stripe payment processing, invoice generation
- **Food & Drinks** - Concession stand management and ordering
- **Real-time Updates** - SignalR for live seat availability and chat
- **Event Management** - Special events and promotions
- **Analytics** - Booking analytics and reporting
- **File Management** - MinIO integration for media storage
- **Email Service** - Automated email notifications via Resend
- **AI Integration** - Gemini API for chatbot functionality
- **QR Code Generation** - Ticket verification system

Prerequisites

- [.NET 8 SDK](#)
- [Docker Desktop](#)
- [PostgreSQL](#)

- [Redis](#)
- [Git](#)

Technology Stack

Backend

- **.NET 8** - Web API framework
- **Entity Framework Core 8** - ORM for database operations
- **PostgreSQL** - Primary database
- **Redis** - Caching and session management
- **SignalR** - Real-time communication

Third-Party Services

- **Stripe** - Payment processing
- **MinIO** - Object storage for files and media
- **Resend** - Email service
- **Google Gemini API** - AI chatbot functionality

Key NuGet Packages

- `Microsoft.AspNetCore.Authentication.JwtBearer` - JWT authentication
- `Microsoft.EntityFrameworkCore.Design` - EF Core design-time tools
- `Npgsql.EntityFrameworkCore.PostgreSQL` - PostgreSQL provider
- `StackExchange.Redis` - Redis client
- `Stripe.net` - Stripe payment integration
- `Minio` - MinIO object storage client
- `QRCoder` - QR code generation
- `Resend` - Email service client

Quick Start

1. Clone the Repository

```
git clone <repository-url>
cd MovieTheater.API
```

2. Environment Setup

Create `appsettings.Development.json` in the `MovieTheater.API` folder:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Port=5432;Database=movietheater_db;Username=postgres;Password=postgres",
    "Redis": "localhost:6379,abortConnect=false"
  },
  "JWT": {
    "SecretKey": "your-secret-key-minimum-32-characters",
    "Issuer": "MovieTheater_Issuer",
    "Audience": "MovieTheater_Audience"
  },
  "GEMINI_API_KEY": "your-gemini-api-key",
  "RESEND_API_TOKEN": "your-resend-api-token",
  "RESEND_FROM": "noreply@yourdomain.com",
  "MINIO_ENDPOINT": "localhost:9000",
  "MINIO_HOST": "http://localhost:9000",
  "MINIO_ACCESS_KEY": "minioadmin",
  "MINIO_SECRET_KEY": "minioadmin",
  "Stripe": {
    "SecretKey": "sk_test_your_stripe_secret_key",
    "PublishableKey": "pk_test_your_stripe_publishable_key"
  }
}
```

3. Using Docker (Recommended)

```
# Start all services
docker-compose up -d

# View logs
docker-compose logs -f

# Stop services
docker-compose down
```

This starts:

- MovieTheater API on `http://localhost:5000`

- PostgreSQL database on localhost:5433
- Redis cache on localhost:6380

4. Manual Setup

Start dependencies:

```
# PostgreSQL
docker run --name movietheater-postgres -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres

# Redis
docker run --name movietheater-redis -p 6379:6379 -d redis:latest
```

Restore and run:

```
dotnet restore
cd MovieTheater.API
dotnet ef database update --project ../MovieTheater.Domain
dotnet run
```

Configuration

Environment Variables

Variable	Description
ConnectionStrings__DefaultConnection	PostgreSQL connection string
ConnectionStrings__Redis	Redis connection string
JWT__SecretKey	JWT signing key
GEMINI_API_KEY	Google Gemini API key
RESEND_API_TOKEN	Resend email service token
MINIO_ENDPOINT	MinIO server endpoint
MINIO_ACCESS_KEY	MinIO access key
MINIO_SECRET_KEY	MinIO secret key

Variable	Description
Stripe__SecretKey	Stripe secret key

API Documentation

Access Swagger UI at: <http://localhost:5000/swagger>

Key Endpoints

Endpoint	Description
POST /api/auth/login	User authentication
POST /api/auth/register	User registration
GET /api/movies	Get all movies
POST /api/bookings	Create booking
GET /api/showtimes	Get showtimes
POST /api/payments/create-checkout-session	Create Stripe payment

Testing

```
# Run all tests
dotnet test

# Run specific test project
dotnet test MovieTheater.UnitTest/
```

VPS Deployment

Prerequisites for VPS

- Ubuntu 20.04+ or CentOS 8+
- Docker and Docker Compose installed
- Domain name pointing to your VPS IP

- SSL certificate (recommended)

1. Server Setup

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER

# Install Docker Compose
sudo curl -L "https://github.com/docker/compose/releases/download/v2.20.0/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
```

2. Deploy Application

```
# Clone repository
git clone <your-repository-url>
cd MovieTheater.API

# Create production environment file
cp docker-compose.yml docker-compose.production.yml
```

Edit docker-compose.production.yml :

services:

movietheater.api:

image: ch1mple/movietheater-api:latest

restart: unless-stopped

environment:

- ASPNETCORE_ENVIRONMENT=Production
- ConnectionStrings__DefaultConnection=Host=postgres;Port=5432;Database=movietheater_db;U:
- ConnectionStrings__Redis=redis:6379,abortConnect=false
- GEMINI_API_KEY=your_production_gemini_key
- JWT__SecretKey=your_production_jwt_secret_key_64_characters_minimum
- RESEND_API_TOKEN=your_production_resend_token
- MINIO_ENDPOINT=your_vps_ip:9000
- MINIO_HOST=https://minio.yourdomain.com
- Stripe__SecretKey=sk_live_your_stripe_live_key

ports:

- "5000:5000"

depends_on:

- postgres
- redis

postgres:

image: postgres:15

restart: unless-stopped

environment:

POSTGRES_USER: postgres

POSTGRES_PASSWORD: your_secure_password

POSTGRES_DB: movietheater_db

volumes:

- postgres_data:/var/lib/postgresql/data

ports:

- "5432:5432"

redis:

image: redis:latest

restart: unless-stopped

volumes:

- redis_data:/data

minio:

image: minio/minio:latest

restart: unless-stopped

environment:

MINIO_ROOT_USER: your_minio_user

```
MINIO_ROOT_PASSWORD: your_minio_password
volumes:
  - minio_data:/data
ports:
  - "9000:9000"
  - "9001:9001"
command: server /data --console-address ":9001"
```

```
volumes:
  postgres_data:
  redis_data:
  minio_data:
```

3. Start Services

```
# Start all services
docker-compose -f docker-compose.production.yml up -d

# Check logs
docker-compose -f docker-compose.production.yml logs -f

# Run database migrations
docker-compose -f docker-compose.production.yml exec movietheater.api dotnet ef database update
```

4. MinIO Configuration

Access MinIO console at http://your_vps_ip:9001

1. Login with your MinIO credentials
2. Create bucket named `movietheater-files`
3. Set bucket policy to public read for movie images
4. Configure DNS/subdomain for MinIO (optional)

5. Nginx Reverse Proxy (Optional)

Install Nginx:

```
sudo apt install nginx
```

Configure `/etc/nginx/sites-available/movietheater` :


```

server {
    server_name movietheaterapi.ae-tao-fullstack-api.site;

    location / {
        proxy_pass          http://127.0.0.1:5001;
        proxy_http_version  1.1;
        proxy_set_header    Upgrade $http_upgrade;
        proxy_set_header    Connection "keep-alive";
        proxy_set_header    Host $host;
        proxy_set_header    X-Real-IP $remote_addr;
        proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto $scheme;
        proxy_cache_bypass  $http_upgrade;
    }

    location /hubs/ {
        proxy_pass          http://127.0.0.1:5001/hubs/;
        proxy_http_version  1.1;
        proxy_set_header    Upgrade $http_upgrade;
        proxy_set_header    Connection "Upgrade";
        proxy_set_header    Host $host;
        proxy_set_header    X-Real-IP $remote_addr;
        proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto $scheme;
        proxy_cache_bypass  $http_upgrade;
    }

    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/movietheaterapi.ae-tao-fullstack-api.site/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/movietheaterapi.ae-tao-fullstack-api.site/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem; # managed by Certbot

}

server {
    if ($host = movietheaterapi.ae-tao-fullstack-api.site) {
        return 301 https://$host$request_uri;
    } # managed by Certbot

    listen 80;
    server_name movietheaterapi.ae-tao-fullstack-api.site;

```

```
    return 404; # managed by Certbot  
}
```

Enable site:

```
sudo ln -s /etc/nginx/sites-available/movietheater /etc/nginx/sites-enabled/  
sudo nginx -t  
sudo systemctl restart nginx
```

6. SSL Certificate (Recommended)

```
# Install Certbot  
sudo apt install certbot python3-certbot-nginx  
  
# Get certificate  
sudo certbot --nginx -d yourdomain.com -d minio.yourdomain.com
```

7. Firewall Configuration

```
# Configure UFW  
sudo ufw allow ssh  
sudo ufw allow 'Nginx Full'  
sudo ufw allow 5000  
sudo ufw allow 9000  
sudo ufw enable
```

8. Monitoring and Maintenance

Create systemd service for auto-restart:

```
# Create service file  
sudo nano /etc/systemd/system/movietheater.service
```

```
[Unit]
Description=MovieTheater API
After=docker.service
Requires=docker.service

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/path/to/MovieTheater.API
ExecStart=/usr/local/bin/docker-compose -f docker-compose.production.yml up -d
ExecStop=/usr/local/bin/docker-compose -f docker-compose.production.yml down
TimeoutStartSec=0

[Install]
WantedBy=multi-user.target
```

Enable service:

```
sudo systemctl enable movietheater.service
sudo systemctl start movietheater.service
```

Troubleshooting

Common Issues

Database Connection

```
docker-compose logs postgres
docker exec -it movietheater_postgres_1 psql -U postgres -d movietheater_db
```

Redis Connection

```
docker-compose logs redis
docker exec -it movietheater_redis_1 redis-cli ping
```

Application Logs

```
docker-compose logs movietheater.api
```

Port Conflicts

```
sudo netstat -tulpn | grep :5000  
sudo lsof -i :5000
```

Security Considerations

- Use strong passwords for all services
- Enable firewall and close unnecessary ports
- Keep Docker images updated
- Use environment variables for secrets
- Enable SSL/HTTPS in production
- Regular database backups
- Monitor application logs

Support

For issues and questions:

- Check application logs
- Review configuration files
- Verify all services are running
- Check firewall and network settings